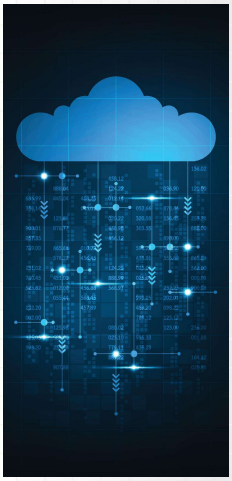


Cloud programming

Rafał Palak

Wrocław University of Science and Technology

1

Creating a Simple Configuration

- Create a configuration file with a .tf extension

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/finding-an-ami.html

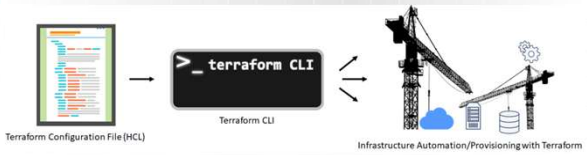
```
provider "aws" {
  region = "us-west-2"
}

resource "aws_instance" "example" {
  # Uwaga: Sprawdź najnowsze AMI dla Twojego regionu https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/finding-an-ami.html
  ami           = "ami-"
  instance_type = "t2.micro"
}
```

2

2

Configuration Files

- Used for **defining and configuring resources** to be managed.
- Terraform **uses the HashiCorp Configuration Language** (HCL) or **JSON** for its configuration files.
- Typically have a **.tf extension** and can be grouped in directories.



3

3

File Structure – Blocks

- HCL files are organized into blocks.
- Block elements for the Block elements for the resource:
  - **aws_instance** - Resource type. In this case, it refers to an EC2 instance in AWS.
  - **Example** - The resource name in the Terraform configuration. It acts as an identifier that allows Terraform to uniquely identify the resource within a given project.
  - **ami = "ami-06dd92ecc74fdfb36"** - An argument setting the Amazon Machine Image (AMI), which is the machine image used to launch the instance.
  - **instance_type = "t2.micro"** - An argument specifying the EC2 instance type. In this case, it's t2.micro

```
resource "aws_instance" "example" {
  ami           = "ami-06dd92ecc74fdfb36"
  instance_type = "t2.micro"
}
```

4

4

AMI - Amazon Machine Image

- **A ready-to-use virtual machine image** that can be used to quickly launch instances (virtual servers).
- It contains all the necessary information to start the machine:
  - **Operating system:** The base operating system that will run on the instance.
  - **Server software:** Server software, such as a web server or database, that is pre-configured in the AMI.
  - **Settings and configurations:** Additional settings and configurations can be stored in the AMI, allowing for the rapid deployment of pre-configured systems.
- **Different AWS regions may have different AMI IDs for the same software.**

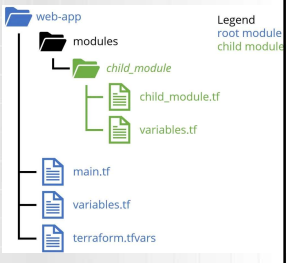
```
resource "aws_instance" "example" {
  ami           = "ami-06dd92ecc74fdfb36"
  instance_type = "t2.micro"
}
```

5

5

Modules

- They are **reusable containers** for Terraform resources that **allow for grouping resources** into logical units that can be **used as building blocks to construct infrastructure**. This **makes the code more organized**, **easier to manage**, and **readily reusable**.
- Utilizing modules **helps avoid code duplication**, **facilitates managing complex configurations**, **promotes best practices in code reusability**, and **simplifies version control** and **updates**.

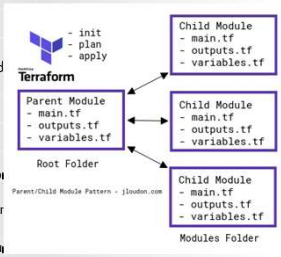


6

6

Modules – how to use

- **Module Definition -** A module is defined by creating a new directory and placing Terraform configuration files within it. Each module should have its own configuration files that define the resources the module will manage.
- **Using a Module -** To use a module, you must add a module block to the main Terraform configuration, specifying the path to the module directory and any inputs (variables) that the module requires.
- **Input and Output Parameters -** Modules can accept input parameters (variables) that allow fo configuring their operation. Modules can also return output values, which can be used by other parts of your Terraform configuration.
- **Public and Private Modules -** You can create you own modules or use modules available publicly in the Terraform Module Registry, where the community shares modules for various providers and services.



7

7

Modules – best practices

- **Detailed Documentation -** Each module should be documented, describing its purpose, required input parameters, and output values.Zakres Modułu
- **Module Scope -** Each module should be responsible for one logical part of the infrastructure, facilitating reuse and management.
- **Versioning** - It is recommended to use versioning for modules, especially when they are shared with others. This allows for better dependency and update management.
- **Modules are a key element in building scalable and manageable configurations in Terraform. They enable the construction of more complex infrastructures in an efficient manner.**

8

8

Modules – example

```
resource "aws_instance" "example" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name = "ExampleInstance"
  }
}
```

```
variable "instance_type" {
  description = "The type of EC2 instance"
  type        = string
}

variable "ami_id" {
  description = "The ID of the AMI for the EC2 instance"
  type        = string
}
```

```
# Specify the provider, in this case, AWS
provider "aws" {
  region = "us-west-2"
}

# Use the module to create an EC2 instance
module "example_ec2_instance" {
  source = "./modules/ec2_instance"

  instance_type = "t2.micro"
  ami_id        = "ami-0c55b159cbfafe1f0"
}
```

9

9

Wrocław University of Science and Technology

Terraform
Variables and Outputs

10

10

Variable [1]

- **Variables** in Terraform **are used to store values** that can be used in multiple places within the configuration. They allow for the parametrization of the configuration, making it easy to adjust resources without having to change the entire code.
- **Variable Types** - Terraform supports various types of variables, including **string**, **number**, **bool**, as well as complex types like **list**, **map**, and **object**.
- **Variable Declaration -** Variables are declared in the configuration files using the variable block, specifying their name and optionally a default value and description.
- **Assigning Values** - Variable values can be assigned on the command line when running Terraform, in configuration files, or in external files, such as .tfvars files or environment variables.

```
variable "instance_type" {
  description = "Typ instancji EC2"
  type        = string
  default     = "t2.micro"
}
```

11

11

Variable [2]

- Variable - defines variables that can be used in the configuration, allowing for its parametrization.
- Elements of the **variable** block:
  - **region -** The name of the variable, which can be used in other parts of the configuration. This allows for the parametrization of various configuration elements.
  - **default = "us-west-2" -** The default value for the variable. If a value for this variable is not provided during the execution of terraform apply or terraform plan, the default value will be used.
  - **Type -** Specifies the type of the variable (e.g., string, list, map, etc.).
  - **Description -** Adds a description of the variable, which is displayed when the user is prompted for its value.
  - **Validation -** Allows for adding validation rules for the variable.

```
variable "region" {
  type        = string
  description = "The AWS region where resources will be created"
  default     = "eu-central-1"
  validation {
    condition     = length(var.region) > 0
    error_message = "The region variable must not be empty."
  }
}
```

12

12

Variable [3]

- This variable can be **used in other parts of the configuration**, facilitating changes.
- Changing the Value - The value of a variable can be changed in several ways, for example, by using the -var flag on the command line, through a terraform.tfvars file, or through environment variables.
- **Planning and Applying -** With terraform plan and terraform apply, Terraform replaces instances of var.region with the actual value of the variable, allowing for dynamic configuration generation.
- Advantages:
  - **Flexibility -** Allows for changing configurations without the need to change the code itself, which is particularly useful in multi-environment setups.
  - **Readability and Maintenance -** Using variables makes the code easier to understand and maintain.
  - **Reusability -** You can use the same variables in different modules and projects, making them highly reusable.

```
provider "aws" {
  region = var.region
}
```

13

13

Variable – validation [1]

- Types of Validation Conditions:
  - **Value Range** - You can specify minimum, maximum values for numeric input variables, or specific values that are allowed for string type variables.
  - **String Format -** Using regular expressions, you can require that string values meet a specific format, such as an email address or UUID.
  - **String Length** - Checking the minimum or maximum length of strings.
  - Verification of Lists and Maps - The ability to check whether lists or maps contain specific elements, or whether their size meets certain requirements.
- Ways to Express Conditions:
  - **Use of Built-in Functions -** Terraform offers a range of functions that can be used to formulate validation conditions, such as length, regex, contains, etc.
  - **Conditional Logic** - You can use logical operators (such as &&, ||, !) to create more complex validation conditions.

```
variable "instance_type" {
  type        = string
  description = "EC2 instance type"

  validation {
    condition     = length(var.instance_type) > 4
    error_message = "The instance_type value must be longer than 4 characters."
  }
}
```

14

14

Variable – validation [2]

- Collaboration and Documentation:
  - **Document Validation Requirements** - Helps other team members understand what values are acceptable and why certain restrictions have been introduced.
  - **Collaborate on Defining Validation -** Engaging team members in the process of defining validation can help capture various use cases and business needs.

```
variable "instance_type" {
  type        = string
  description = "EC2 instance type"

  validation {
    condition     = length(var.instance_type) > 4
    error_message = "The instance_type value must be longer than 4 characters."
  }
}
```

15

15

Locals [1]

- **Locals** - are used to define local variables that are available only within a specific configuration file or module. They are useful for simplifying configurations and for enhancing the readability of the code.
- Elements of the **locals** block:
  - ***common_tags -*** *This is the name of the local variable. You can name it anything you wish.*
  - ***Owner = "devops team" -*** *This is a key-value pair for the map assigned to* ***common_tags****. You can have multiple such key-value pairs in one local variable.*
- The local variable **common_tags** is **accessible within the same configuration file** and **can be used in various blocks**, like resource, module, provider, etc.

```
locals {
  common_tags = {
    Owner = "devops team"
  }
}
```

16

16

Locals [2]

- Pros:
  - **Readability -** Allows for the grouping of repetitive configuration elements, making the code more organized.
  - **Reusability -** With local variables, it's easier to manage code that is to be used in multiple places within a single configuration.
  - **Context Preservation -** Local variables are only visible within the file/module, which helps in maintaining context and limits the scope of variables to places where they are actually needed.
  - **Composition -** Enables the assembly of different configuration elements from smaller, easier to manage pieces, such as common tags.

```
locals {
  common_tags = {
    Owner = "devops team"
  }
}
```

17

17

locals vs variable

Locals	Variable
• **Scope -** Local variables are only available within a single configuration file or module where they are defined. • **No Modifications -** They cannot be modified using command line arguments or variable files (*.tfvars). • **Readability** - Primarily used to simplify code and improve readability within a single file or module. • **No Default Values and Validation** - It is not possible to specify default values or use validation. • **No User Interaction** - The end user does not interact with them directly.	• **Scope -** They can be used throughout the entire project and can be passed between modules. • **Modification** - They can be modified using command line arguments, variable files (*.tfvars), or environmental variables. • **Reusability** - Used to create configurations that can be easily adapted to different environments or settings. • **Default Values and Validation** - It is possible to specify default values, data types, and even implement validation. • **User Interaction** - The end user often provides values for the variables, allowing for customization of Terraform's behavior.

18

18

Output [1]

- Outputs in Terraform are used to return information about resources created by Terraform, which can be useful for the user or other parts of the Terraform configuration.
- Outputs are declared using the output block, specifying their name and value.
- Outputs can be used to obtain important information after applying the configuration, such as IP addresses, resource IDs, etc. They are particularly useful in modules, where the output from one module can be passed as input to another.

```
output "instance_id" {
  value       = aws_instance.example.id
  description = "The ID of the created instance."
}
```

```
module "source_module" {
  source        = "./modules/source_module"
  instance_name = "ExampleInstance"
}

module "target_module" {
  source               = "./modules/target_module"
  source_instance_id = module.source_module.instance_id
}
```

```
variable "source_instance_id" {
  type        = string
  description = "The source instance ID to be used in this module."
}
```

19

19

Output [2]

- **Output -** Defines the outputs that will be displayed after applying the configuration. They are used to extract values from the created infrastructure.
- Elements of the output block:
  - **instance_ip - This is the name of the output value. It is used to identify this particular output value in the results of the terraform apply command and to refer to it in other places.**
  - **value = aws_instance.my_instance.public_ip -** Specifies the value that will be assigned to instance_ip.
  - **Description -** A description of the output value, which can be helpful for others using the configuration.
  - **Sensitive** - A setting that specifies whether the output value contains sensitive data and should be hidden in logs.

```
output "instance_ip" {
  value       = aws_instance.my_instance.public_ip
  description = "The public IP address of the instance"
  sensitive   = false
}
```

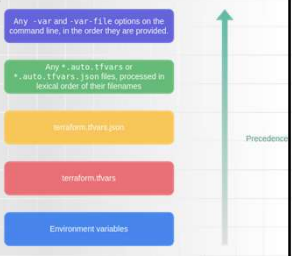
```
resource "aws_instance" "my_instance" {
  ami           = "ami-06dd92ecc74fdfb36"
  instance_type = "t2.micro"
}
```

20

20

Variables and Outputs – good practices

- **Variable Naming -** It is recommended to use meaningful names for variables that clearly define what they contain and what they are used for.
- **Organization -** Group related variables and outputs to facilitate configuration management, especially in larger projects.
- **Modularization -** Variables and outputs can be helpful in creating modules that can be easily reused in different projects and environments.
- **Security -** It is advisable to be cautious when outputting sensitive data, especially in multi-access or public environments.



21

21

Wrocław University of Science and Technology

Terraform

the most important types of blocks

22

22

Blocks – module [1]

- **Module -** is used for utilizing modules, which are reusable, standalone pieces of Terraform code.
- Modules are units that can be parameterized, used in different projects, and even shared with the community. They are particularly useful in large projects and teams, where various elements of infrastructure are created in a similar manner.

```
module "my_vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "2.64.0"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["us-west-2a", "us-west-2b", "us-west-2c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.4.0/24", "10.0.5.0/24", "10.0.6.0/24"]

  enable_nat_gateway = true
  enable_vpn_gateway = true

  tags = {
    Terraform   = "true"
    Environment = "dev"
  }

  providers = {
    aws = aws.custom
  }

  depends_on = [
    aws_iam_role.example
  ]
}
```

23

23

Blocks – module [2]

- **source = "terraform-aws-modules/vpc/aws“ -** Indicates that the module code will be downloaded from the terraform-aws-modules repository for VPC in AWS.
- **version = "2.64.0“ -** Specifies the version of the module, ensuring consistency and stability.
- **name = "my-vpc“ -** The name of the VPC that will be created.
- **cidr = "10.0.0.0/16“ -** The CIDR range for the VPC. It specifies what IP addresses will be available in this private cloud.
- **azs = ["us-west-2a", "us-west-2b", "us-west-2c"] -** The list of Availability Zones where the subnets will be located.
- **private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"] -** The list of CIDR ranges for private subnets.
- **public_subnets = ["10.0.4.0/24", "10.0.5.0/24", "10.0.6.0/24"] -** The list of CIDR ranges for public subnets.

```
module "my_vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "2.64.0"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["us-west-2a", "us-west-2b", "us-west-2c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.4.0/24", "10.0.5.0/24", "10.0.6.0/24"]

  enable_nat_gateway = true
  enable_vpn_gateway = true

  tags = {
    Terraform   = "true"
    Environment = "dev"
  }

  providers = {
    aws = aws.custom
  }

  depends_on = [
    aws_iam_role.example
  ]
}
```

24

24

Blocks – module [3]

- **enable_nat_gateway = true -** This option causes a NAT (Network Address Translation) gateway to be created in the VPC.
- **enable_vpn_gateway = true -** This option enables the creation of a VPN gateway.
- **tags = { ... } -** A map of tags that will be attached to the resources created by this module.
- **providers = { aws = aws.custom } -** Specifies that the module will use a custom AWS provider named aws.custom.
- **depends_on = [aws_iam_role.example] -** Ensures that the module will not be executed until a specified IAM role (aws_iam_role.example) is created.

```
module "my_vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "2.64.0"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["us-west-2a", "us-west-2b", "us-west-2c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.4.0/24", "10.0.5.0/24", "10.0.6.0/24"]

  enable_nat_gateway = true
  enable_vpn_gateway = true

  tags = {
    Terraform   = "true"
    Environment = "dev"
  }

  providers = {
    aws = aws.custom
  }

  depends_on = [
    aws_iam_role.example
  ]
}
```

25

25

Blocks –resource

- **Resource** - is used to define resources that are to be managed by Terraform.

```
resource "aws_instance" "example" {
  ami           = "ami-06dd92ecc74fdfb36"
  instance_type = "t2.micro"
}
```

26

26

Resource

- **A resource is a unit of infrastructure, such as a virtual machine, security group, or database.**
- **Declaration -** Resources are declared in Terraform configuration files.
- **Attributes -** Each resource has various attributes that can be configured. For example, for a virtual machine in AWS (EC2), you can set the instance type, SSH key, image, etc.
- **Dependencies -** Resources can have dependencies on each other. For example, a resource representing a database may require that a resource representing a VPC network be created first.
- **Variables -** Variables can be used to parameterize resources, increasing the flexibility and reusability of the configuration.
- **State -** Information about resources is stored in state files, allowing Terraform to manage their lifecycle.

```
provider "aws" {
  region = "eu-central-1"
}

resource "aws_instance" "moja_instancja" {
  ami           = "ami-06dd92ecc74fdfb36"
  instance_type = "t2.micro"

  tags = {
    Name = "moja_instancja"
  }
}
```

27

27

Resource - example

- **The resource is an EC2 instance in Amazon Web Services (AWS).**
- **Provider - The provider "aws" defines the use of AWS as the cloud service provider.**
- **region = "us-west-2" specifies the region where the resources will be created.**
- **resource "aws_instance" "my_instance" - the resource we want to create. It defines a resource of type aws_instance with the name my_instance.**
- **Attributes - Within the resource block, various attributes are set:**
  - **ami = "ami-0abcdef1234567890" -** Specifies the AMI ID that will be used to create the instance.
  - **instance_type = "t2.micro" -** The type of instance being created.
  - **tags = { Name = "my_instance" } -** Labels assigned to the instance.

```
provider "aws" {
  region = "eu-central-1"
}

resource "aws_instance" "moja_instancja" {
  ami           = "ami-06dd92ecc74fdfb36"
  instance_type = "t2.micro"

  tags = {
    Name = "moja_instancja"
  }
}
```

28

28

Blocks – provider

- **Provider -** specifies the service provider with which Terraform will interact, e.g., AWS, Azure, Google Cloud.
- Elements of the provider block:
  - **aws**: This is the type of provider, in this case, Amazon Web Services. There are many available providers such as Azure, Google Cloud, DigitalOcean, etc.
  - **region = "us-west-2"**: This is a configuration argument specifying the AWS region in which Terraform will manage resources.
  - **profile**: Specifies the AWS CLI profile to be used for authentication.
  - **access_key** i **secret_key**: Authentication keys if you are not using an AWS CLI profile or IAM role.
  - **version**: You can define the version of the provider you want to work with.

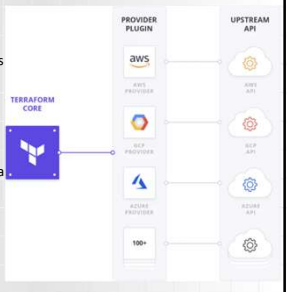
```
provider "aws" {
  region     = "eu-central-1"
  profile    = "my-aws-profile"
  version    = "~> 2.0"
}
```

29

29

Providers [1]

- **Definition -** Providers are services or platforms with which Terraform interacts in order to manage resources. The most popular ones include AWS, Azure, and Google Cloud, but Terraform supports many others, including on-premises providers like VMware.
- **Multiple Providers -** It's possible to use multiple providers in a single project, allowing for the management of resources across different clouds or services.
- **Configuration -** Each provider has its own set of required and optional configuration arguments. These arguments are typically used for authorization and configuring various settings.
- **Resources and Attributes -** Each provider offers a set of resources and their attributes that can be managed. For example, the AWS provider offers resources such as aws_instance for EC2 or aws_s3_bucket for S3.



30

30

Providers [2]

- **Documentation** - Each provider has its own documentation that describes the available resources, their attributes, and how to configure them. This is essential for effectively using Terraform with a given provider.
- **Initialization** - To use a specific provider, it needs to be initialized in the project using the terraform init command.

31

31

Providers - example

- **provider "aws"** tells Terraform to manage resources uaing **Amazon Web Services (AWS)** provider.

```
provider "aws" {
  region = "eu-central-1"
}

resource "aws_instance" "moja_instancja" {
  ami           = "ami-06dd92ecc74fdfb36"
  instance_type = "t2.micro"

  tags = {
    Name = "moja_instancja"
  }
}
```

32

32

Wroclaw University of Science and Technology

Thank you for your attention

33

33